Workshop 5: Actor Critic - Final Report

Authors: Zhengnan Cao, Yuchen Yang

This report answers Task 1 and all four Task 2 questions using 17 formal conditions. Each condition uses the preselected training seed 0: AC/GAE requests 500,000 steps and SAC runs for 1,000,000 steps. Every 10,000 training steps, the stochastic policy is evaluated for ten fixed-seed episodes (10000-10009), independently of training. Evaluation steps are excluded from the horizontal axis. The same SAC baseline is reused across questions.

Tables distinguish the final evaluation from the mean of the last five checkpoints. Tail SD is the population standard deviation across those five checkpoint means, not across training seeds. AUC is the time-averaged return over the recorded interval (10k-1M for SAC); first return 1000 is a descriptive speed diagnostic, not a convergence criterion. SAC training scalars are counted 1000-update window means, including the shorter final window; plots add no smoothing. One seed supports descriptive comparisons only.

Late scalar summaries (alpha, entropy, Q, targets and critic MSE) use logged SAC windows ending in the final 50k-step region (950k-1M), weighted by each window's actual update count, window_updates.

Task 1 - Question 1: HalfCheetah environment and policies

Question

What are the state space, action space and reward structure? Are they finite or infinite, discrete or continuous? Is the environment deterministic or stochastic, and are the AC and SAC policies deterministic or stochastic?

Answer

Component	Description and classification
State / observation	17 continuous coordinates: eight generalized positions (global horizontal position excluded) and nine velocities. The observation Box is unbounded and contains infinitely many values.
Action	Six continuous joint-torque controls in [-1, 1]. Bounded but with infinitely many possible values.
Reward	Continuous scalar: forward velocity minus 0.1 times the sum of squared actions. Its set of possible values is infinite.

With default settings, forward velocity is (x_next - x)/0.05. Given the complete simulator state and action, MuJoCo physics is deterministic. Reset adds uniform position noise and Gaussian velocity noise with scale 0.1, so episodes start stochastically. HalfCheetah has no task termination by default; TimeLimit truncates episodes at 1000 steps.

Both policies are stochastic during training and evaluation. AC samples a Gaussian action, clipping only the value sent to the environment; its log probability uses the original sample. SAC samples a Gaussian and applies tanh. Fixed evaluation seeds make sampled episodes reproducible without making the policies deterministic.

Task 1 - Question 2: AC/GAE training on HalfCheetah

Question

Train AC with GAE (lambda=0.99) on HalfCheetah-v5. Log progress with TensorBoard, submit evaluation returns and actor loss versus environment steps, and comment on training success.

Submit

Configuration: 500k steps, gamma=0.99, Actor/Value LR=0.001, two 64-unit hidden layers, batches of at least 1000 steps and normalized advantages. GAE recurses backward within each episode. Only true termination suppresses bootstrap; either termination or truncation stops the recursion. The value target is advantage plus the current value estimate.

$$\delta_t = r_t + \gamma(1 - \text{terminated}_t)V(s_{t+1}) - V(s_t)$$

$$\hat{A}_t = \delta_t + \gamma\lambda(1 - \text{done}_t)\hat{A}_{t+1}, \qquad \hat{R}_t = \hat{A}_t + V(s_t)$$

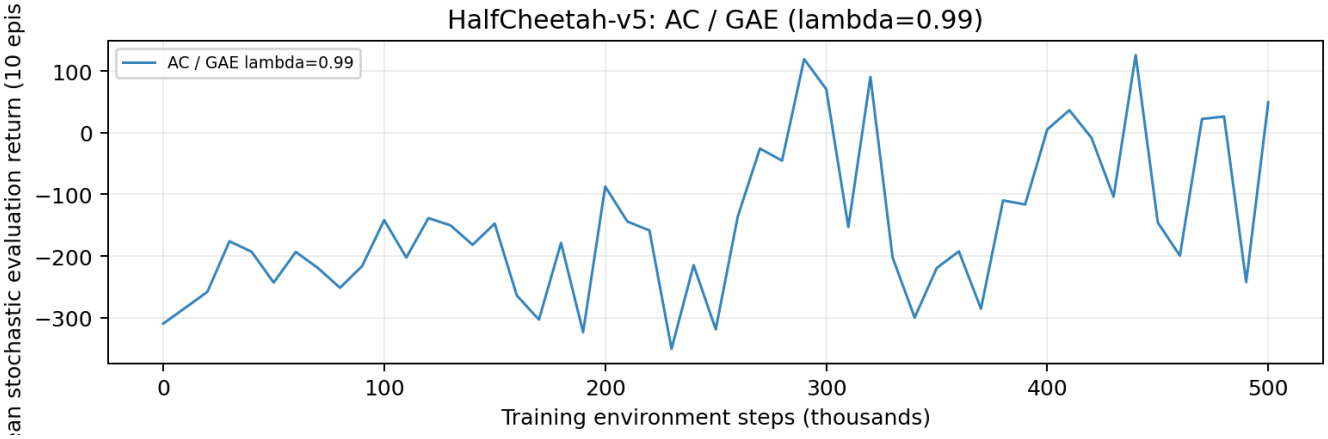


Figure 1. AC/GAE evaluation returns versus training environment steps.

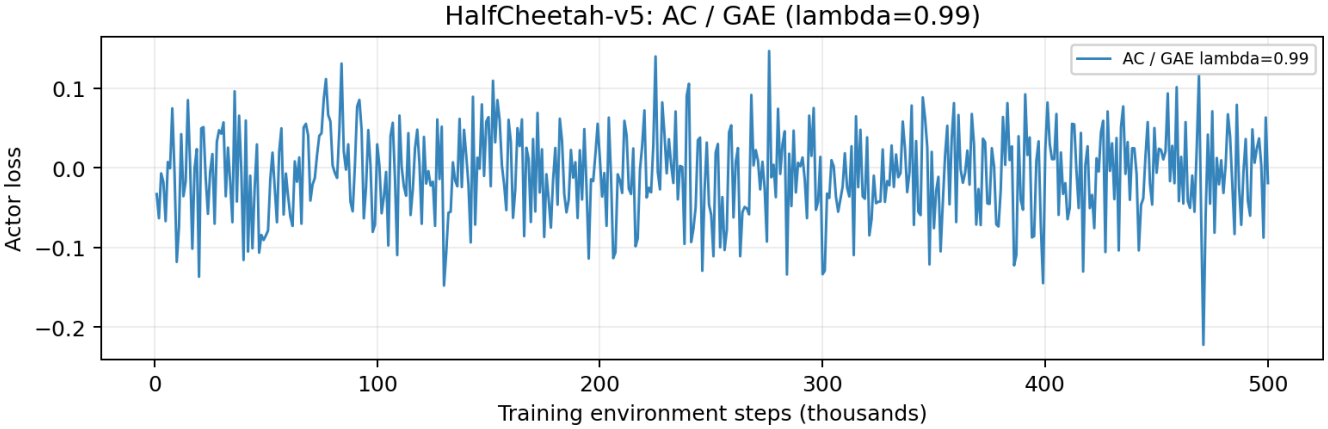


Figure 2. AC actor loss, recorded once per update batch.

Discussion

Does the learning curve suggest successful training?

Return improves from -309.9 at the first checkpoint to 49.7 at the last one, but the last-five mean is -68.7 and tail SD is 125.5. The frequent drops and non-positive late mean do not demonstrate stable successful learning. The positive final checkpoint alone is insufficient; actor-loss sign or decrease is not a success criterion.

Task 1 - Question 3: SAC training on HalfCheetah

Question

Train SAC on HalfCheetah-v5, visualize progress with TensorBoard and submit evaluation-return and actor-loss curves. Check positive return by 50k steps and continued improvement beyond 500k steps.

Submit

Baseline: gamma=0.99, Actor/Critic LR=1e-4, two 256-unit hidden layers, initial alpha=0.2 with automatic temperature tuning (target entropy -6, temperature LR=3e-4), tau=0.005, replay capacity 1M, batch=256 and warm-up=5000. One update follows each transition after warm-up. The target uses the smaller target critic and an entropy term; the actor uses reparameterized actions with the tanh log-probability correction.

$$y = r + \gamma(1 - \text{terminated})[\min_{i=1,2} \bar{Q}_i(s\prime, a\prime) - \alpha \log \pi_\theta(a\prime \mid s\prime)]$$

$$J_\pi = \mathbb{E}_{s \sim \mathcal{D},\, a \sim \pi_\theta(\cdot \mid s)}[\alpha \log \pi_\theta(a \mid s) - \min_{i=1,2} Q_i(s, a)]$$

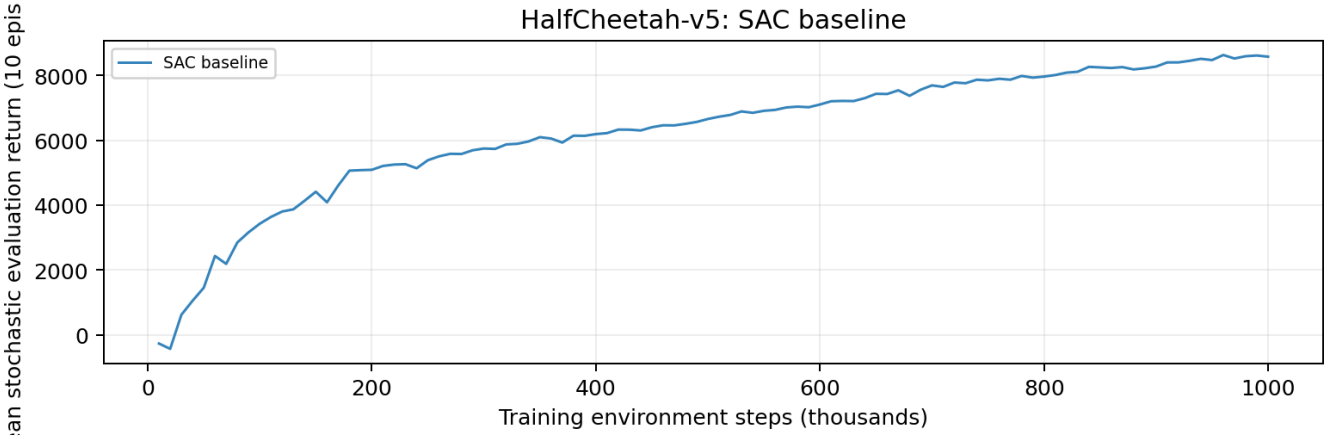


Figure 3. SAC baseline evaluation returns.

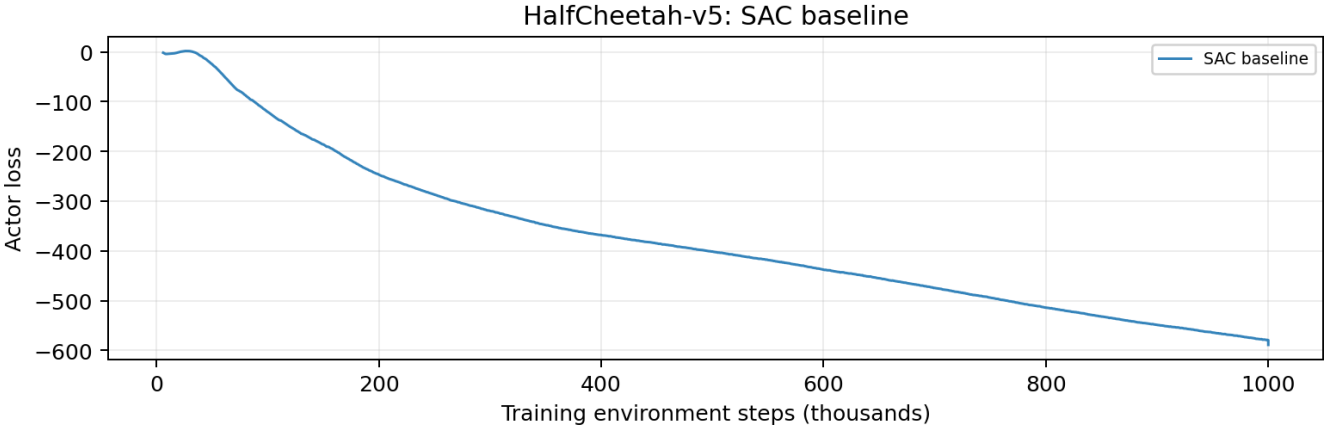


Figure 4. SAC actor loss; counted update-window means.

Discussion

Does SAC meet the expected learning behavior?

Yes, in the observed seed-0 run. Return first becomes positive at 30,000 steps and is 1,453.9 at 50k. It reaches 6,657.2 at 500k and 8,578.0 at 1M; the last-five mean is 8,587.7. Mean return rises from 6,509.3 over 450k-500k to 8,568.7 over 950k-1M. This supports useful learning and further late improvement, while not proving that no temporary plateau occurs or that the result generalizes across training seeds.

TensorBoard logging was enabled. The submitted plots reconstruct the CSV scalar records, whose values were checked against the event files; this verification does not constitute actual browser inspection.

Task 2 - Question 1: Effect of GAE lambda

Question

Compare at least three lambda values in [0.9, 0.99] on InvertedPendulum-v5. Submit evaluation-return curves and discuss observed behavior, the lambda=0 and lambda=1 estimators, and the bias-variance trade-off.

Submit

Values are 0.90, 0.95 and 0.99. Other AC settings match Task 1; each run requests 500k steps and finishes complete episodes, explaining the small budget overshoots below.

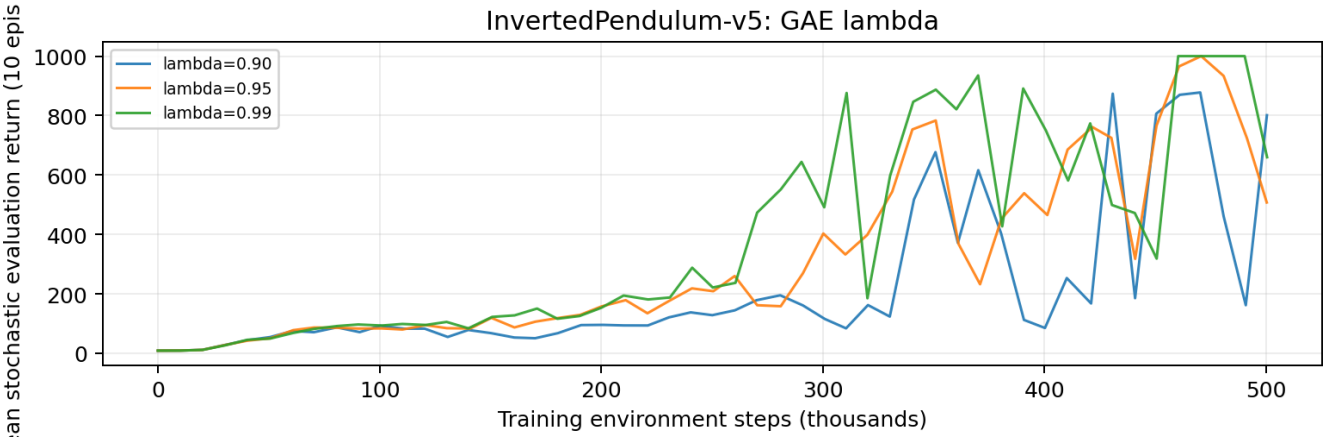


Figure 5. InvertedPendulum evaluation returns for all three GAE values.

Lambda	Actual steps	Final return	Last-five mean	Tail SD
0.90	500,295	801.1	634.1	281.4
0.95	500,233	507.5	826.0	186.1
0.99	500,376	659.7	931.9	136.1

Discussion

What behavior is observed for different lambda values?

Lambda=0.99 has the highest last-five mean (931.9), followed by 0.95 (826.0) and 0.90 (634.1). The 0.95 and 0.99 runs first reach reference return 950 near the 460k evaluation; their few-hundred-step difference is smaller than the 10k evaluation interval. All three have late drops, so these results favor 0.99 for late performance without establishing stable convergence.

What do lambda=0 and lambda=1 correspond to?

Lambda=0 gives the one-step TD residual. Lambda=1 sums all discounted TD residuals: at a truly terminating episode boundary this telescopes to Monte Carlo return minus V(s); finite time-limit truncation retains a bootstrap term. These are theoretical endpoints; the experiments use only 0.90, 0.95 and 0.99.

How does lambda affect bias, variance and performance?

Smaller lambda relies more on the critic, usually reducing variance but increasing sensitivity to value-estimation bias. Larger lambda uses longer reward paths, usually reducing bootstrap bias while increasing variance. The better late returns at 0.99 are consistent with a useful trade-off here; return variation is not a direct measurement of gradient-estimator variance.

Task 2 - Question 2: Initial entropy temperature

Question

Compare initial alpha in [0.01, 1] with automatic temperature tuning and other settings fixed. Submit evaluation-return and actor-loss curves; discuss exploration, speed, stability, possible policy collapse and the entropy objective.

Submit

Initial values are 0.01, 0.2 and 1.0; the Task 1 SAC baseline supplies 0.2. These are initial temperatures, not constant entropy weights throughout training.

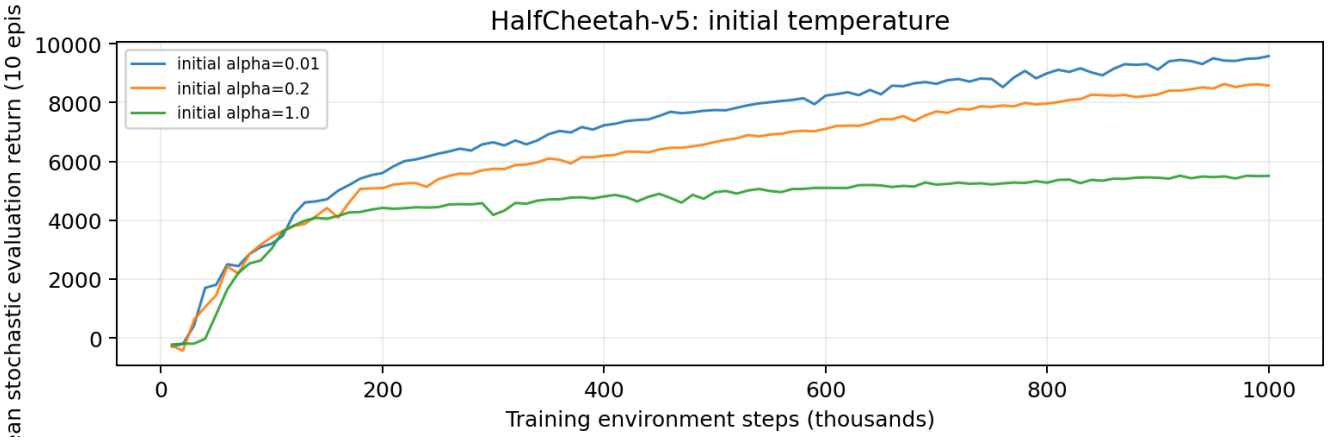


Figure 6. Evaluation returns for three initial temperatures.

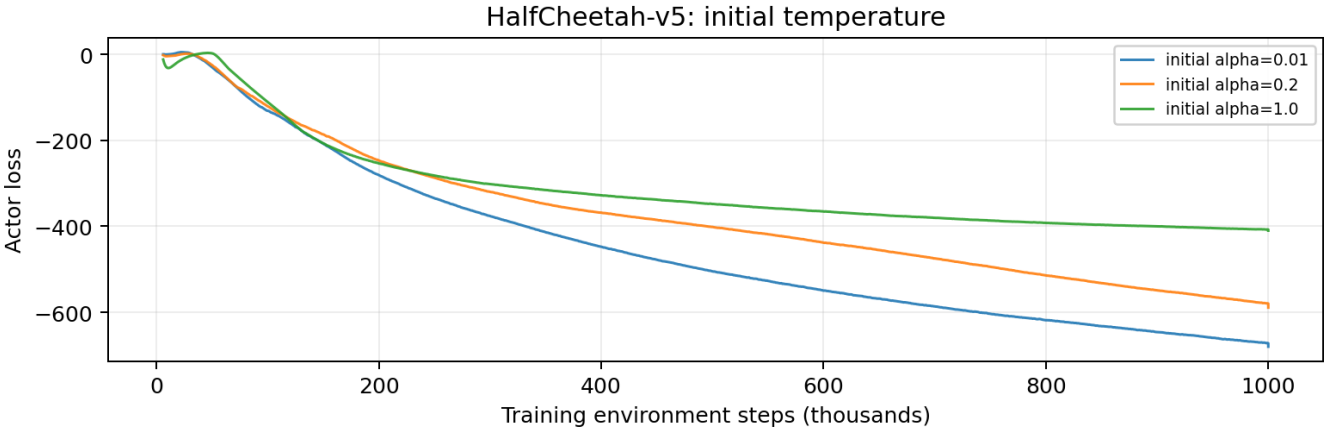


Figure 7. Actor loss for three initial temperatures.

Initial alpha	50k return	Last-five mean	Tail SD	Late alpha
0.01	1,809.2	9,478.2	59.0	0.246
0.2	1,453.9	8,587.7	37.1	0.263
1.0	803.7	5,483.9	33.3	0.137

Discussion

How do lower and higher initial temperatures affect exploration, speed and stability?

Higher alpha initially gives entropy a larger weight, encouraging exploration while potentially delaying exploitation. Here 0.01 and 0.2 first become positive at 30k, while 1.0 does so at 50k. Their last-five means are 9478.2, 8587.7 and 5483.9: the largest initial temperature learns more slowly and finishes lower in this seed. Automatic tuning later changes all three temperatures, but does not remove the influence of the early data-collection path.

For late stability, alpha=0.01 has tail SD 59.0, compared with 37.1 for 0.2: its higher return accompanies more checkpoint fluctuation. Alpha=1.0 has the smallest tail SD (33.3), but also the lowest return. Lower SD therefore does not establish a better policy; this measures local return variation, not gradient variance or across-seed stability.

Does a small alpha cause premature policy collapse?

It can weaken the exploration incentive in principle, but the 0.01 run attains the highest return of the three and its late entropy estimate is near the target -6. This run therefore does not support inevitable collapse at 0.01. The late entropy estimates of all three are near -6; negative differential entropy is valid for a continuous distribution and is not itself collapse. Action-coverage or multi-seed diagnostics would be needed for a stronger claim.

How does this relate to the SAC entropy objective?

The actor minimizes alpha*log pi - min(Q1,Q2), balancing reward-seeking against entropy. Automatic tuning targets entropy rather than maximizing return directly. Initial exploration and replay contents can produce different final returns even after entropy estimates become similar. Actor-loss magnitude combines the entropy and Q terms, so it cannot independently rank policy quality.

Task 2 - Question 3: Actor and critic learning rates

Question

Sweep actor and critic learning rates within [1e-5, 1e-2]. Submit evaluation returns, actor loss and both critic losses. Discuss each tested pair and identify the fastest useful learning without destabilization.

Submit

Five one-factor-at-a-time pairs test the low and high endpoints around the 1e-4/1e-4 baseline. This is not a full two-dimensional grid. All other settings and the 1M-step budget are fixed. Critic losses use a symlog axis (linear below 1) to show the near-zero and larger losses together.

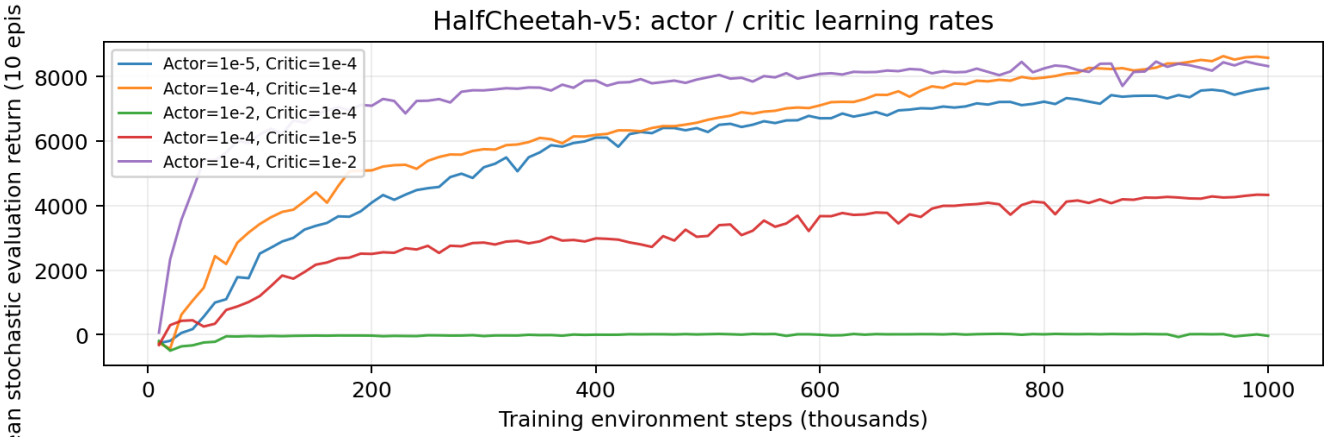


Figure 8. Learning-rate evaluation returns.

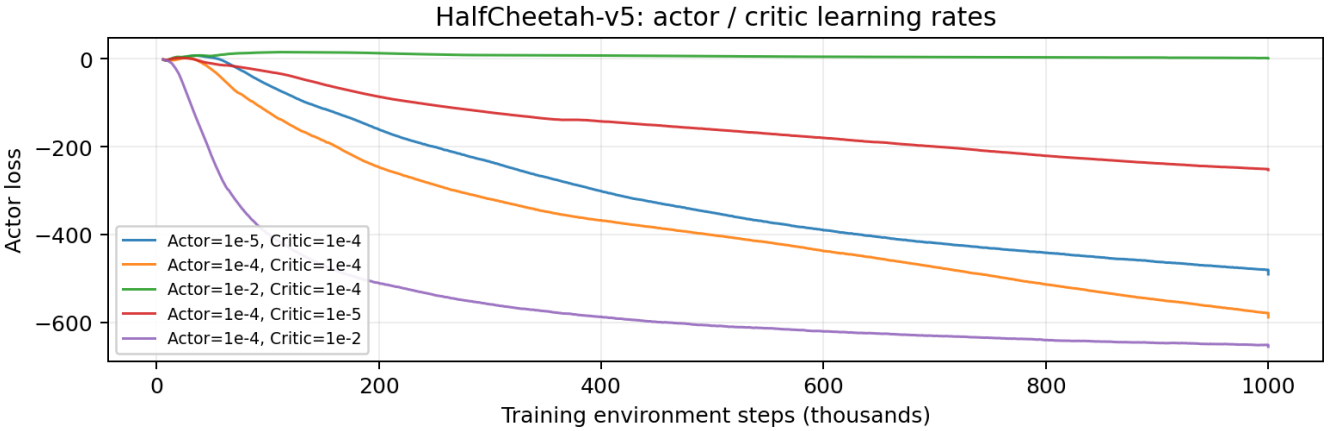


Figure 9. Learning-rate actor losses.

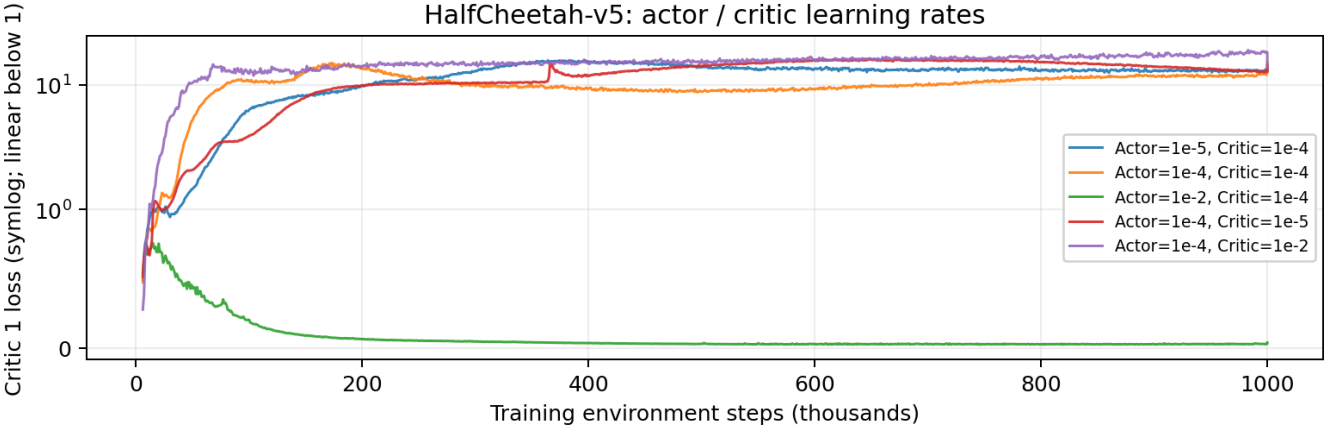


Figure 10. Learning-rate Critic 1 losses.

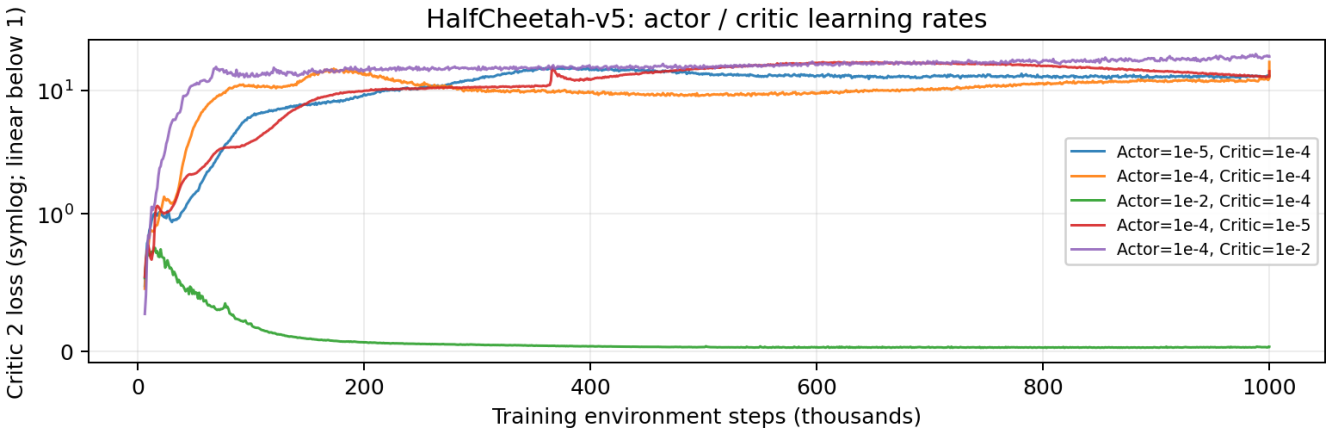


Figure 11. Learning-rate Critic 2 losses.

Actor LR	Critic LR	Last-five mean	Tail SD	AUC	First 1000
1e-5	1e-4	7,547.5	69.8	5,640.4	70,000
1e-4	1e-4	8,587.7	37.1	6,298.8	40,000
1e-2	1e-4	-18.5	27.2	-23.7	not reached
1e-4	1e-5	4,296.0	35.3	3,082.9	90,000
1e-4	1e-2	8,390.4	55.9	7,533.4	20,000

Discussion

Which pair learns fastest without destabilizing?

Actor=1e-4/Critic=1e-2 is the strongest speed candidate here: it first reaches return 1000 at 20k and has the highest AUC (7533.4), versus 40k and 6298.8 for the baseline. It completes 1M steps with finite losses and a last-five mean of 8390.4. The baseline has the higher late mean (8587.7) and lower tail SD (37.1 versus 55.9). Thus the high critic rate improves early sample efficiency with useful late performance, while the baseline is better on the late-return measure. Neither establishes strict or cross-seed convergence.

What happens in each tested case?

Actor=1e-5, Critic=1e-4. Reaching return 1000 takes 70k steps, and the last-five mean is 7547.5. Relative to the baseline, the smaller actor step delays improvement; the late curve still rises, so this is slower learning rather than numerical divergence.

Actor=1e-4, Critic=1e-4. This baseline has the highest last-five mean among these pairs. Its mean return increases by 2059.4 between the 450k-500k and 950k-1M windows, so it remains useful late in training without evidence that it has completely stabilized.

Actor=1e-2, Critic=1e-4. The last-five mean is -18.5 despite finite late Critic 1/2 MSEs of 0.029/0.028. Large policy updates can chase inaccurate value estimates; these measurements are consistent with ineffective control, not a measured gradient explosion. Fitting a low-reward replay distribution can give a small MSE without a good policy. This unfavorable result is retained.

Actor=1e-4, Critic=1e-5. It needs 90k steps to reach return 1000 and finishes with last-five mean 4296.0. This slower return growth is consistent with slow critic adaptation. Its small tail SD (35.3) does not offset the low return or prove lower gradient variance.

Actor=1e-4, Critic=1e-2. Fastest early growth, slightly lower late return than the baseline. Its late critic MSEs are 17.955/18.627, versus 11.847/12.062 for the baseline; finite MSE alone cannot rank policies with different replay distributions.

Task 2 - Question 4: Replay size, batch size and learning start

Question

Compare replay capacities {100k, 500k, 1M}, batches in [32, 512] and learning starts {1k, 5k, 10k}. Submit evaluation returns, Q values and target values for each; discuss adaptation, overfitting, variance, stability and sample efficiency.

Submit

Each factor changes alone from replay=1M, batch=256 and start=5k. Batches are 32, 256 and 512; all conditions run for 1M environment steps. Q is the mean min(Q1,Q2) at replay states with newly sampled current-policy actions. Target is the mean entropy-regularized bootstrap target for replay transitions. These use different actions and constructions; their mean difference is not critic MSE.

Replay capacity

Setting	Last-five mean	Tail SD	AUC	Late Q	Late target
Replay=100k	9,947.3	397.2	7,481.9	782.2	782.8
Replay=500k	9,420.5	63.2	6,502.6	714.5	714.0
Replay=1M	8,587.7	37.1	6,298.8	573.7	572.7

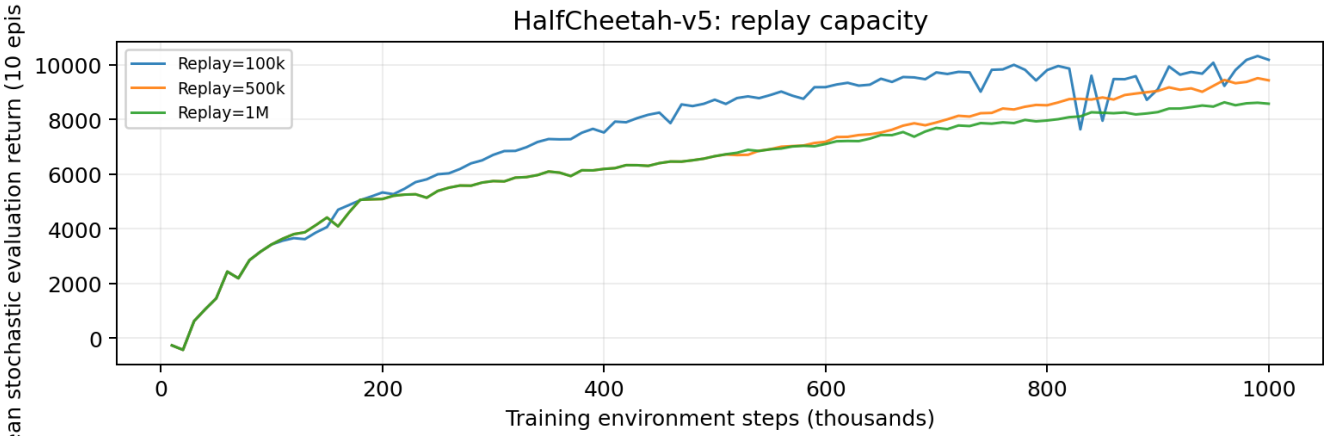


Figure 12. Replay capacity: evaluation returns.

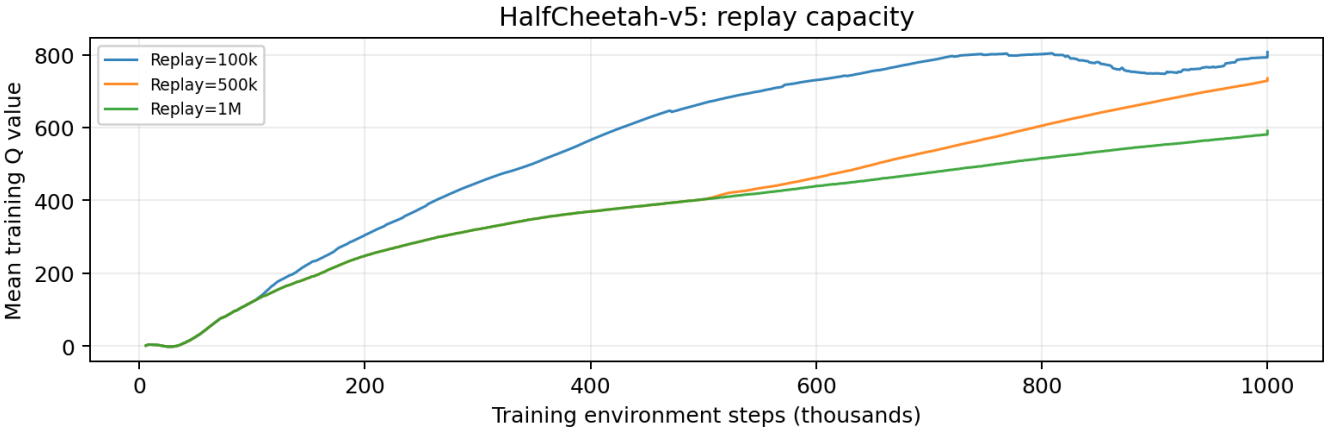


Figure 13. Replay capacity: mean training Q values.

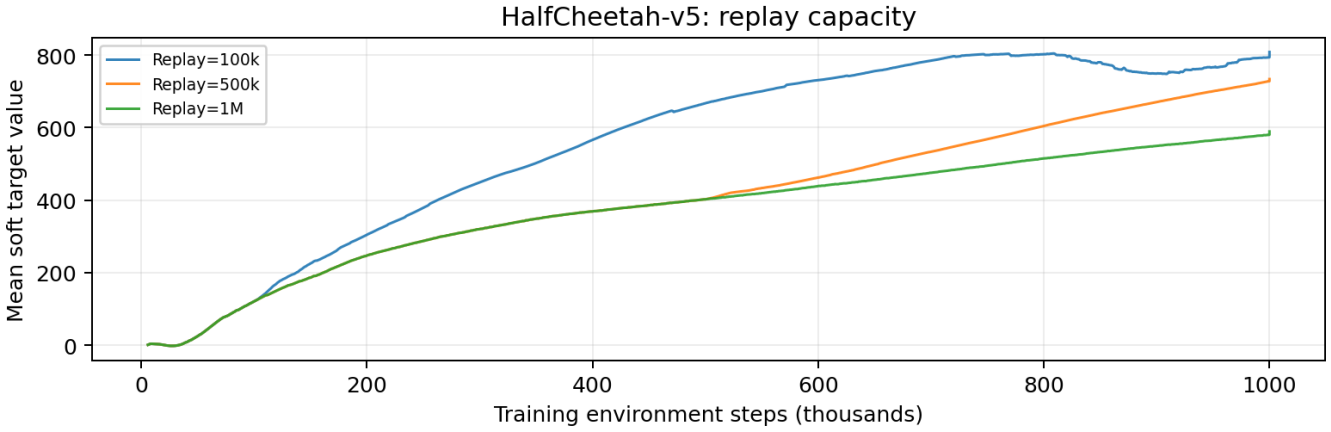


Figure 14. Replay capacity: mean soft targets.

Batch size

Setting	Last-five mean	Tail SD	AUC	Late Q	Late target
Batch=32	5,584.7	10.6	4,314.5	350.4	350.2
Batch=256	8,587.7	37.1	6,298.8	573.7	572.7
Batch=512	7,182.7	51.1	5,575.4	477.0	476.6

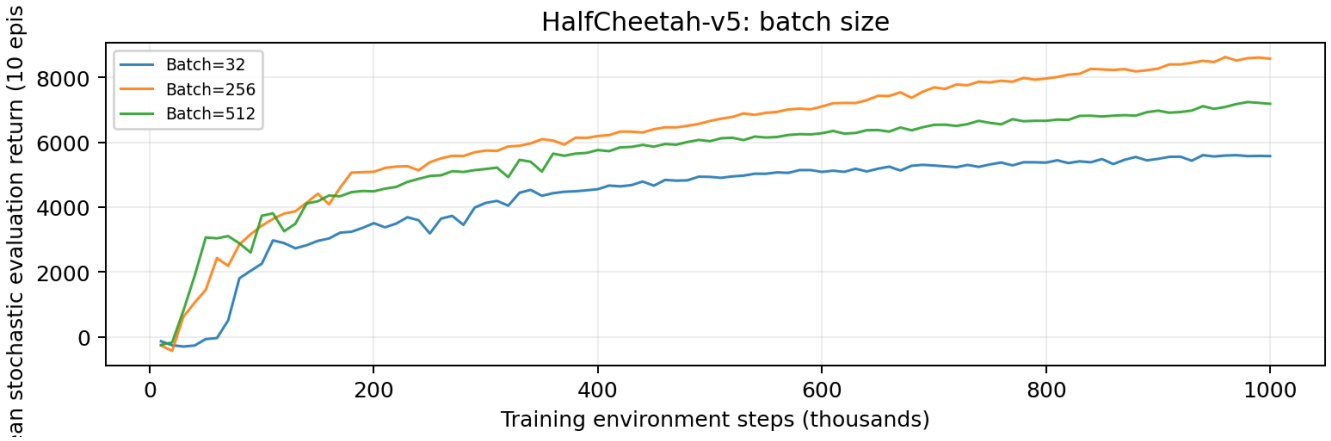


Figure 15. Batch size: evaluation returns.

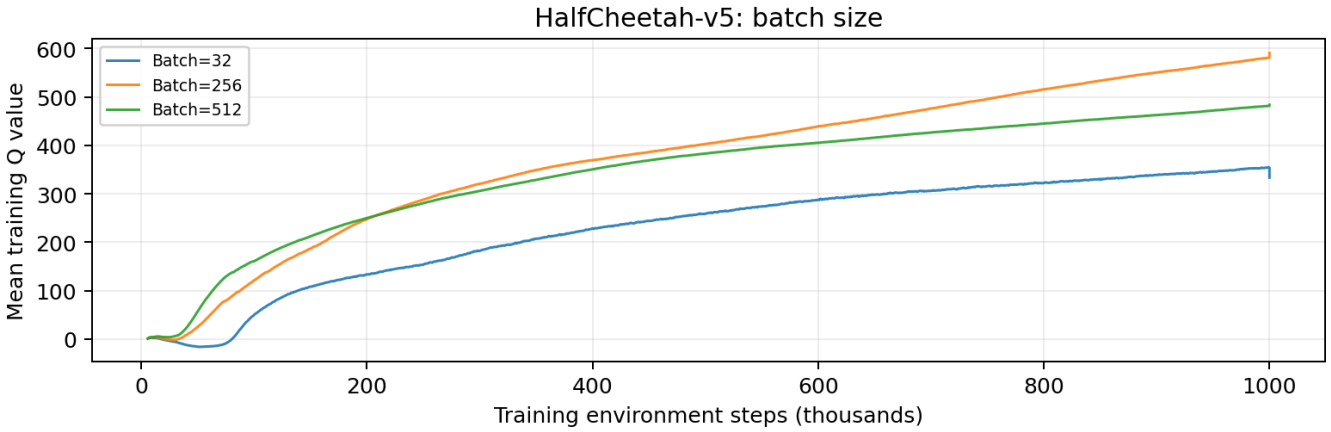


Figure 16. Batch size: mean training Q values.

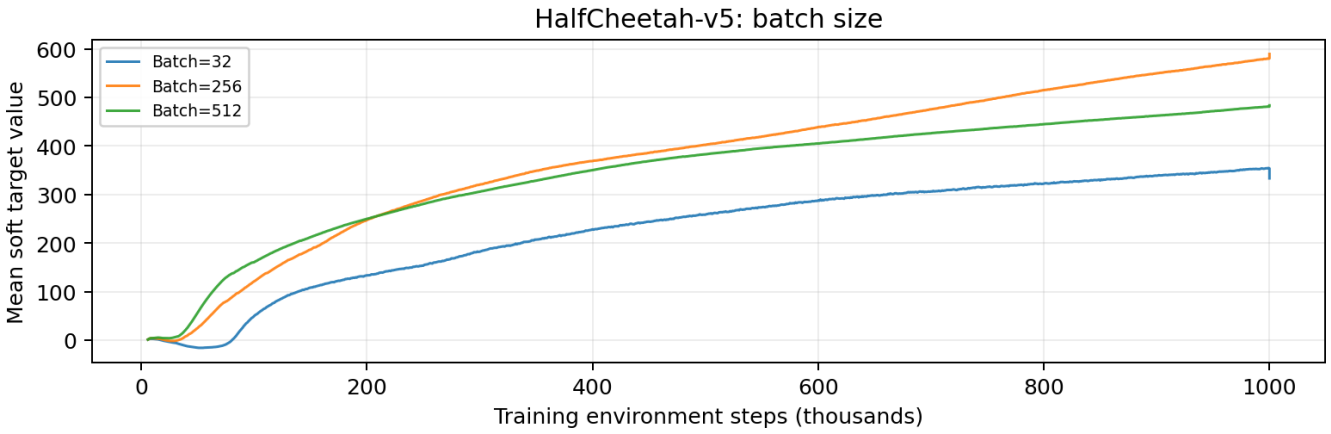


Figure 17. Batch size: mean soft targets.

Learning start

Setting	Last-five mean	Tail SD	AUC	Late Q	Late target
Start=1k	5,420.6	23.4	4,470.7	386.3	386.1
Start=5k	8,587.7	37.1	6,298.8	573.7	572.7
Start=10k	8,181.0	59.1	6,034.1	577.8	576.9

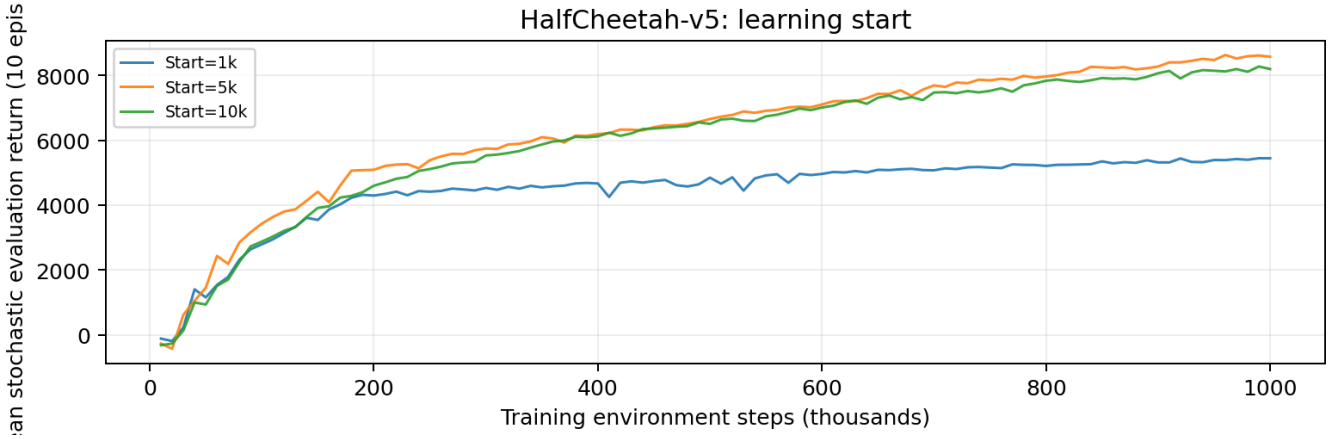


Figure 18. Learning start: evaluation returns.

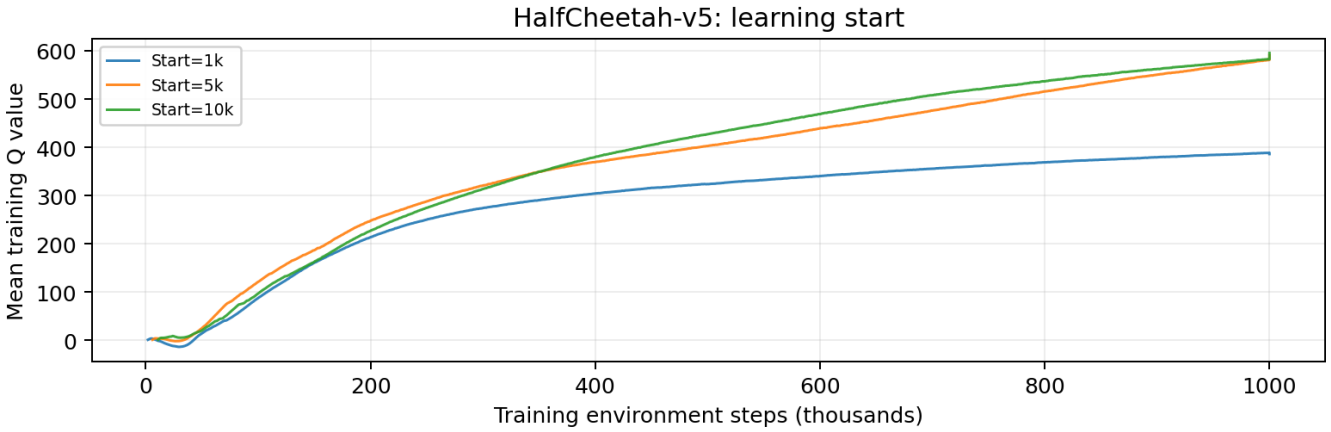


Figure 19. Learning start: mean training Q values.

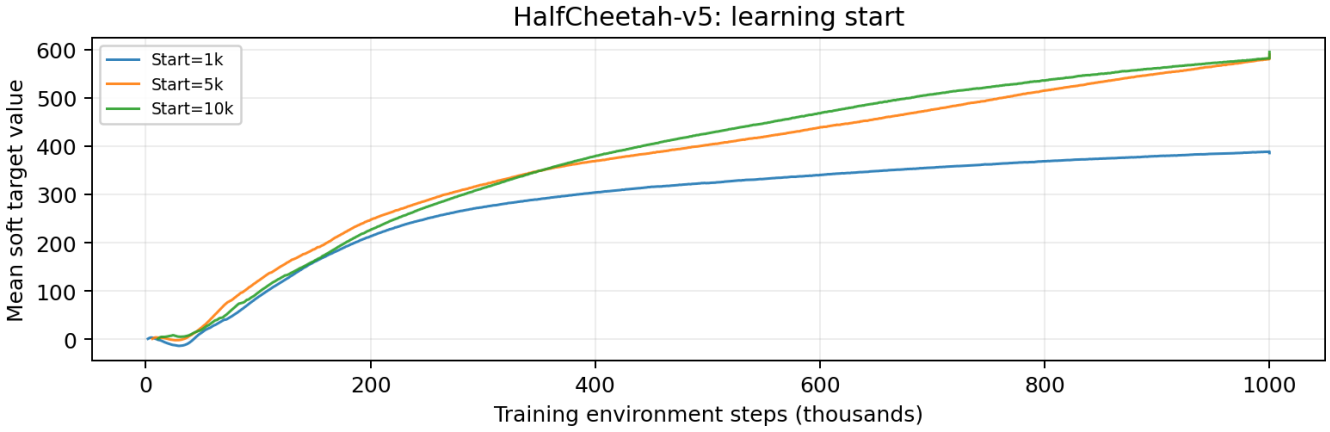


Figure 20. Learning start: mean soft targets.

Discussion

Does a smaller buffer adapt faster but overfit, while a larger buffer stabilizes learning?

Replay=100k achieves the highest last-five return (9947.3), followed by 500k (9420.5) and 1M (8587.7), but their tail SDs are 397.2, 63.2 and 37.1. Smaller replay can replace old-policy data faster, consistent here with higher return and more late fluctuation. This is an adaptation/stability trade-off, not evidence establishing overfitting: no held-out generalization experiment was run.

The 100k and 500k runs exactly match the baseline at their 10 and 50 shared evaluation checkpoints through 100k and 500k steps, respectively, before different eviction behavior starts. All three first reach return 1000 at 40k, so that early threshold cannot distinguish capacity effects. The runs are independent executions with a common fixed-seed prefix; post-eviction behavior is the relevant comparison.

Do larger batches reduce variance and improve stability or sample efficiency?

Batch=256 has the highest late mean (8587.7) and AUC (6298.8) of the three. Batch=512 has better return at 50k (3066.9 versus 1453.9), but finishes lower (7182.7), so its early advantage does not persist. Batch=32 reaches return 1000 at 80k rather than 40k and has last-five mean 5584.7. Its smaller tail SD (10.6) describes a flatter, lower-return trajectory; it is not evidence of smaller gradient variance.

All batches make 995001 updates. Sampling 32, 256 or 512 replay items per update changes replay work, not the number of new environment interactions. Observed process times are 35.7, 40.7 and 44.0 minutes; these include startup and evaluation, so they are separate from environment-sample efficiency and are not pure GPU timings.

Does longer warm-up reduce variance and improve stability?

Not monotonically in this run. Starts 1k/5k/10k give last-five returns 5420.6/8587.7/8181.0 and tail SDs 23.4/37.1/59.1. The 1k and 5k settings reach return 1000 at 40k, while 10k needs 60k; 5k also has the highest AUC. Earlier learning therefore does not automatically improve sample efficiency, and the longest warm-up is not the most stable late policy here.

Longer warm-up increases random experience before learning but delays updates. The update counts are 999001, 995001 and 990001. Differences reflect both the initial data and update timing; no extra ablation isolates these causes. Q and target means stay on comparable scales, but such averages cannot prove accurate value calibration or reduced gradient variance.

Supplementary temperature diagnostics

The following curves support Task 2 Question 2. Late temperature and entropy summaries weight each window by its actual update count, so the final short window is not over-weighted.

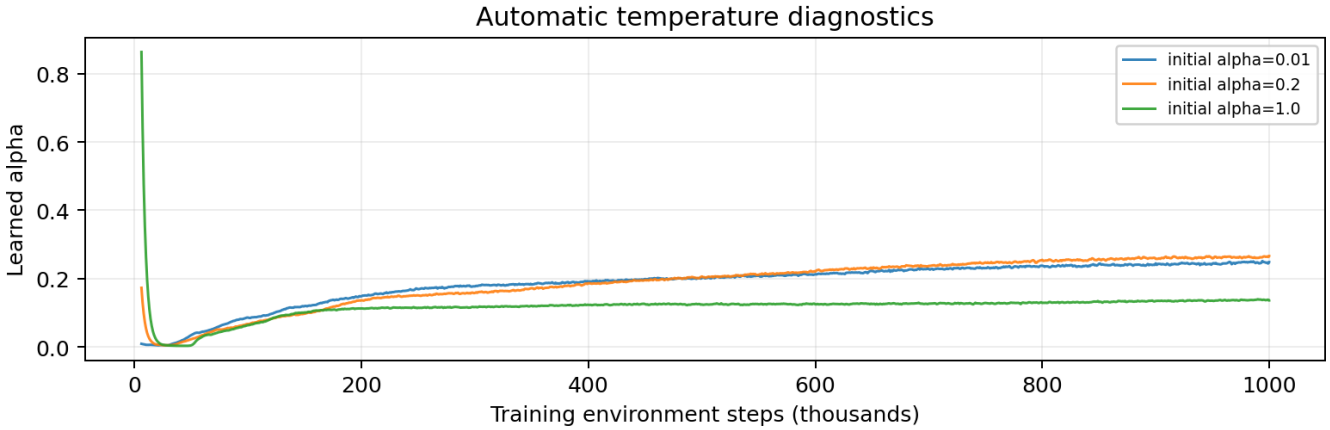


Figure 21. Automatically tuned temperature from each initial alpha.

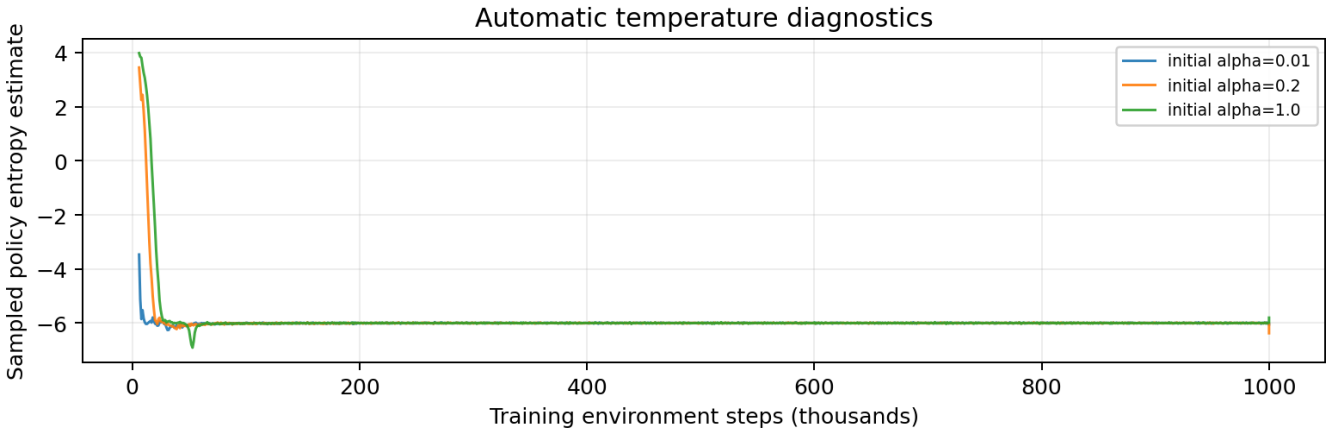


Figure 22. Sampled policy differential entropy; target -6.

References and reproducibility

Provided ELEN90098 Workshop 5 handout, pp. 5-7 (tasks), pp. 2-4 (environment and algorithm definitions). Environment details were checked against the installed Gymnasium 1.3.0 implementation. Runtime, full configurations, frozen source, CSVs, TensorBoard events and checkpoints are retained under evidence; the approved run map and numerical summary are evidence/planning/approved_default_steps_matrix.json and evidence/analysis/formal_results_summary.json. Implementation optimizations preserve one SAC update per transition and include Chinese comments; verification details are separate from the answers above.